

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	K. HEMASREE	Reg. No.:	192411156
Section / Batch:	SLOT C	Date:	23/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I K. HEMASREE(192411156) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

AIM

To design and implement a **Data Center Load Balancing Optimization algorithm using the Divide and Conquer technique**, which distributes incoming requests among multiple servers while considering server capacity and current load.

ALGORITHM

1. **Start** the program.
2. Create a list of servers, each with a unique server ID and maximum capacity.
3. Initialize the current load of every server to **0**.
4. Read the incoming request loads.
5. Divide the request list into two halves recursively.
6. For each request, find the available server with the **least current load** that can handle the request without exceeding its capacity.
7. Assign the request to the selected server.
8. Update the server's current load after assignment.
9. Continue the divide-and-conquer process until all requests are processed.
10. Display the request-to-server assignments and final server loads.
11. **Stop** the program.

INPUT

Example input used in the program:

Incoming Requests:

[10, 15, 5, 20, 25, 10, 30, 5, 15, 20]

The requests represent the amount of workload generated by different incoming requests.

OUTPUT

Incoming Requests: [10, 15, 5, 20, 25, 10, 30, 5, 15, 20]

--- Assigning Requests using Divide & Conquer ---

Request (load=10) -> Server S1
Request (load=15) -> Server S2
Request (load=5) -> Server S3



SIMATS Online Programming Lab
DAA PROGRAMMING

 KEERTHIPATI HEMA SREE
1924111566C

CO4 AT2- Industry Problem Solving Task

← Back

QUESTIONS

Q1

Data Center Load Balancing Optimization (Divide & Conquer)

100 marks

1/1 answered

Q1 Data Center Load Balancing Optimization (Divide & Conquer)100 marks

A data center must distribute incoming requests across multiple servers to maintain optimal performance. Analyze how divide and conquer strategies can be applied to balance load efficiently. Identify constraints such as server capacity and request variability. Design a scalable load balancing algorithm. Discuss real-time implementation challenges. Evaluate performance in terms of latency and throughput. Interpret results in high-performance computing environments.

Your Code

```
1 # Data Center Load Balancing Optimization using Divide & Conquer
2
3 class Server:
4     def __init__(self, server_id, capacity):
5         self.server_id = server_id
6         self.capacity = capacity
7         self.current_load = 0
8
9     def can_handle(self, request_load):
10         return (self.current_load + request_load) <= self.capacity
11
12     def assign(self, request_load):
13         self.current_load += request_load
14
15
16 def divide_and_conquer_balance(requests, servers, low, high):
17     """
18     Recursively divides the list of requests and assigns them
19     to the least loaded available server using divide & conquer.
20     """
21     if low > high:
22         return
23
```

Input

stdin input

Output

Incoming Requests: [10, 15, 5, 20, 25, 10, 30, 5, 15, 20]
--- Assigning Requests using Divide & Conquer ---
Requests (Allocation) -> Server S1